

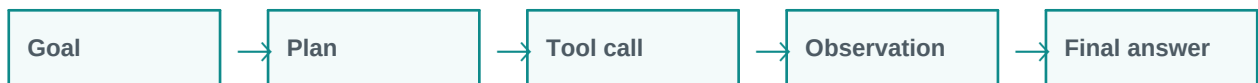


LLM Agents

Agents can plan and act using tools.

Simple explanation: An LLM agent is an AI assistant powered by an LLM that can plan, use tools, take actions, observe results, and complete a goal step by step.

Learning style: this handout starts with a very simple explanation and gradually increases the depth. It is designed for classroom training, participant reading, and quick revision.



What you will learn

- Meaning in simple words
- Key components and architecture
- Practical examples and use cases
- Where it fits in GenAI and agentic AI systems
- Benefits, challenges, and implementation points

1. Beginner-Friendly Explanation

An LLM agent is an AI assistant powered by an LLM that can plan, use tools, take actions, observe results, and complete a goal step by step.

Think of it like this:

An agent is like a smart assistant that can use tools. It does not only speak; it can search, calculate, read files, call APIs, and decide the next step.

Simple real-time examples

- Analyze PDF and create quiz
- Debug code using logs
- Search web and compare products
- Query database and summarize
- Create training material

2. Gradually Deeper Explanation

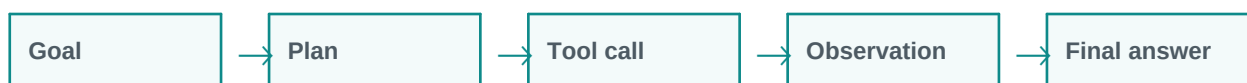
LLM Agents becomes more powerful when we understand its components, data flow, limitations, and business fit. In real projects, the concept is rarely used alone. It usually works with prompts, models, tools, documents, APIs, evaluation, monitoring, and human review.

Important concepts

Concept	Meaning
Research agent	Research agent is an important part of understanding and applying LLM Agents in practical GenAI systems.
Coding agent	Coding agent is an important part of understanding and applying LLM Agents in practical GenAI systems.
Data analysis agent	Data analysis agent is an important part of understanding and applying LLM Agents in practical GenAI systems.
Customer support agent	Customer support agent is an important part of understanding and applying LLM Agents in practical GenAI systems.
Scheduling agent	Scheduling agent is an important part of understanding and applying LLM Agents in practical GenAI systems.

3. Architecture / Flow

A typical architecture can be understood as a simple flow. The exact design changes based on the project, but the pattern below is a useful starting point.



Architecture explanation

Step 1: Goal - This stage contributes to the final outcome. It should be designed carefully because weak inputs, wrong retrieval, poor prompts, or missing validation can reduce quality.

Step 2: Plan - This stage contributes to the final outcome. It should be designed carefully because weak inputs, wrong retrieval, poor prompts, or missing validation can reduce quality.

Step 3: Tool call - This stage contributes to the final outcome. It should be designed carefully because weak inputs, wrong retrieval, poor prompts, or missing validation can reduce quality.

Step 4: Observation - This stage contributes to the final outcome. It should be designed carefully because weak inputs, wrong retrieval, poor prompts, or missing validation can reduce quality.

Step 5: Final answer - This stage contributes to the final outcome. It should be designed carefully because weak inputs, wrong retrieval, poor prompts, or missing validation can reduce quality.

Common implementation layers

- User interface or API layer
- Prompt and instruction layer
- Model layer
- Tool/retrieval/data layer
- Validation, safety, and logging layer
- Human review or feedback layer

4. Detailed Examples

Below are example scenarios showing how this topic appears in real GenAI projects.

Scenario	How it works	Expected output
Analyze PDF and create quiz	The system receives the request, applies LLM Agents, and processes the task step by step.	A useful, structured, reviewable result.
Debug code using logs	The system receives the request, applies LLM Agents, and processes the task step by step.	A useful, structured, reviewable result.
Search web and compare products	The system receives the request, applies LLM Agents, and processes the task step by step.	A useful, structured, reviewable result.
Query database and summarize	The system receives the request, applies LLM Agents, and processes the task step by step.	A useful, structured, reviewable result.



Scenario	How it works	Expected output
Create training material	The system receives the request, applies LLM Agents, and processes the task step by step.	A useful, structured, reviewable result.

5. Benefits and Challenges

Benefits	Challenges / Risks
Improves productivity and reduces repetitive manual effort.	Can produce incorrect or unsupported answers if not validated.
Helps users interact with complex systems in natural language.	Needs careful design of prompts, data access, permissions, and monitoring.
Can support training, support, coding, analysis, and document workflows.	May have cost, latency, privacy, and governance concerns in production.

6. Best Practices

- Start with a simple prototype before building a complex system.
- Use clear prompts and structured output formats.
- Add validation and human review for important decisions.
- Measure quality with realistic test cases, not only demo examples.
- Monitor cost, latency, accuracy, safety, and user feedback.
- Document assumptions, limitations, and escalation paths.

7. Key Takeaways

LLM Agents is an important concept in modern LLM and GenAI systems. The simple idea is easy to understand, but production implementation requires thoughtful architecture, data quality, evaluation, safety, and monitoring.

Quick revision

- Simple meaning: An LLM agent is an AI assistant powered by an LLM that can plan, use tools, take actions, observe results, and complete a goal step by step.
- Architecture: input, processing, tools/data, validation, output.
- Business value: saves time, improves knowledge access, and supports automation.
- Risk control: use grounding, evaluation, guardrails, monitoring, and human approval.